

0.29.1. Revenir au départ avec `va_a` — de base, ailleurs

Le 0.29 dix lignes plus bas : le B va en (10, 15) et revient en (0, 10).

1. Ce qu'il faut comprendre

C'est le 0.29, avec une autre lettre, le **B** (`ALPHABET[1]`, la 2e lettre), et une autre place : le **départ (0, 10)**, l'arrivée (10, 15), et le retour en **(0, 10)**.

C'est la **version de base** : une seule lettre, à la deuxième place. Le 0.29.2 met les deux ensemble.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <va_a>` → [0.27.3. La fonction `va\_a\(\)` — aller jusqu'à une case](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1125 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      63 o ← la console, à l'allumage - la mise en route de la console
va_a()          176 o ← main() - une fonction fournie par le langage
main()          64 o ← démarrage - ta fonction
VBlank          8 o ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage   28 o ← image() - attend l'image suivante
AttendreVBlank  16 o ← démarrage - attend que l'écran soit libre
AttendreAcces   12 o ← EffacerCases - attend de pouvoir écrire une case
AdresseCase     10 o ← poser(), effacer() - trouve l'adresse d'une case de l'écran
EffacerCases    12 o ← effacer() - efface des cases
CopierTuiles    18 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte    14 o ← démarrage - vide les deux cartes de l'écran
Tuiles          704 o ← CopierTuiles - la police entière
```

Lettres gravées : toutes (`poser()` reçoit un numéro calculé)

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreFinVBlank`, `EcrireTexte`, `EcrireNombre`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `DonneesTransfert`, `Notes`

Fonctions du langage appelées :

```
poser()    x 2    26 o sur place, dans va_a() + AdresseCase (10 o)
effacer() x 1    15 o sur place, dans va_a() + AdresseCase (10 o), EffacerCases (12 o)
image()    x 2     6 o sur place, dans va_a(), main() + AttendreImage (28 o)
```

Tes fonctions appelées :

```
va_a()    x 2    45 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <va_a>          // mène une tuile jusqu'à une case
#include <ALPHABET>       // les lettres de la police : ALPHABET[0] est le A
```

2. UNE VARIABLE — X

```
uint8_t x = 0;      // la colonne du B
```

3. UNE VARIABLE — Y

```
uint8_t y = 10;     // sa ligne
```

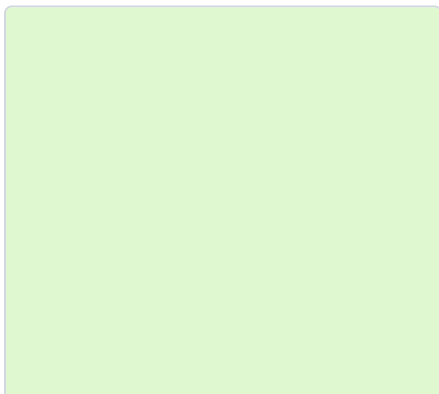
4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {          // Le programme commence ici.
    // va_a(colonne, ligne, tuile, arrivéeX, arrivéeY) : la tuile va jusqu'à la
    // case d'arrivée (en diagonale d'abord, puis tout droit), et la console range
    // l'arrivée dans x et y.
    // Le changement : le B (ALPHABET[1]) part de (0, 10).
    va_a(x, y, ALPHABET[1], 10, 15); // l'aller : jusqu'en (10, 15)
    va_a(x, y, ALPHABET[1], 0, 10);  // le retour : jusqu'en (0, 10), le départ

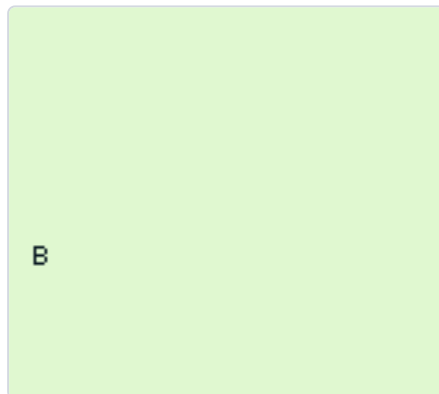
    while (true) {      // La boucle de jeu, pour toujours :
        image();         // attendre l'image suivante.
    }
}
```

3. Ce qui se passe, étape par étape

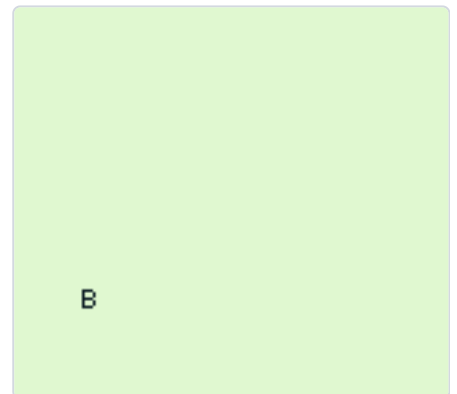
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



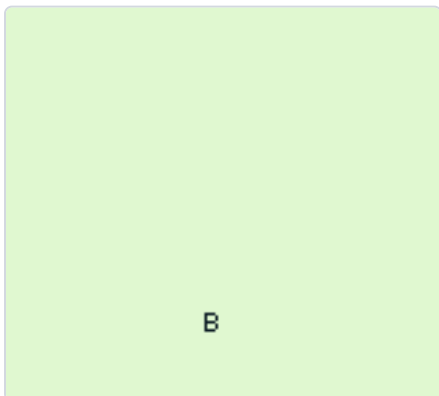
la toute première image — la console vient de démarrer — image 1



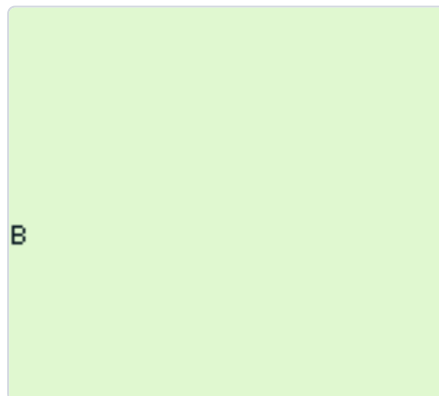
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Un B qui va en (10, 15) et revient en (0, 10).

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le B passe par (10, 15)

✓ le B finit en (0, 10) (0,10)

Ensuite : **0.29.2. Revenir au départ avec `va_a` — doublé, deux positions** — chapitre 0 : Avant tout. · [gameboy3](#) — écrire une cartouche Game Boy en C++